



ML EXPERIMENT 6

Name : Abhishek Pandey
SAP ID: 60019220110

Subject: Machine Learning

Aim: Implement Clustering.

Code:

```
import networkx as nx
```

```
import matplotlib.pyplot as plt
```

```
# Nodes and edges
```

```
labels = ['A','B','C','D']
```

```
edges = [
```

```
    ('A','B',1),
```

```
    ('A','C',3),
```

```
    ('B','C',2),
```

```
    ('B','D',4),
```

```
    ('C','D',5)
```

```
]
```

```
# Create graph
```

```
G = nx.Graph()
```

```
for u, v, w in edges:
```

```
    G.add_edge(u, v, weight=w)
```



```
# Print input graph edges
```

```
print("Input Graph Edges with Weights:")
```

```
for u, v, w in G.edges(data=True):
```

```
    print(f'{u} -- {v} : {w["weight"]}')

# Positions for consistent layout
```

```
# Positions for consistent layout
```

```
pos = nx.spring_layout(G, seed=42)
```

```
# 1 Plot Input Graph
```

```
plt.figure(figsize=(6,5))
```

```
nx.draw(G, pos, with_labels=True, node_size=500)
```

```
edge_labels = nx.get_edge_attributes(G, 'weight')
```

```
nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels)
```

```
plt.title("Input Graph")
```

```
plt.show()
```

```
# 2 Compute MST
```

```
mst = nx.minimum_spanning_tree(G)
```

```
# 3 Plot MST over full graph for comparison
```

```
plt.figure(figsize=(6,5))
```

```
# Draw full graph edges in light gray
```

```
nx.draw(G, pos, with_labels=True, node_size=500, edge_color='lightgray')
```

```
# Draw MST edges in green
```

```
nx.draw_networkx_edges(mst, pos, width=2, edge_color='green')
```

```
# Draw nodes and labels
```

```
nx.draw_networkx_nodes(G, pos, node_size=500)
```

```
nx.draw_networkx_labels(G, pos)
```

```
# Edge labels only for MST
```



```
mst_edge_labels = nx.get_edge_attributes(mst, 'weight')
mst_edge_labels = {k: f'{v}' for k,v in mst_edge_labels.items()}
nx.draw_networkx_edge_labels(G, pos, edge_labels=mst_edge_labels)
plt.title("MST (green) over Input Graph (gray)")
plt.show()

# 4 Remove largest edge in MST to form clusters
largest_edge = max(mst.edges(data=True), key=lambda x: x[2]['weight'])
mst.remove_edge(largest_edge[0], largest_edge[1])
clusters = list(nx.connected_components(mst))
print("\nClusters after cutting MST:")
for i, c in enumerate(clusters, 1):
    print(f"Cluster {i}: {list(c)}")

# 5 Plot clusters
colors = ['red','blue','green','orange']
plt.figure(figsize=(6,5))
for i, cluster in enumerate(clusters):
    nx.draw_networkx_nodes(mst, pos, nodelist=cluster, node_color=colors[i],
node_size=500)
nx.draw_networkx_edges(mst, pos, width=2)
nx.draw_networkx_labels(mst, pos)
plt.title("Clusters from MST")
plt.show()
```

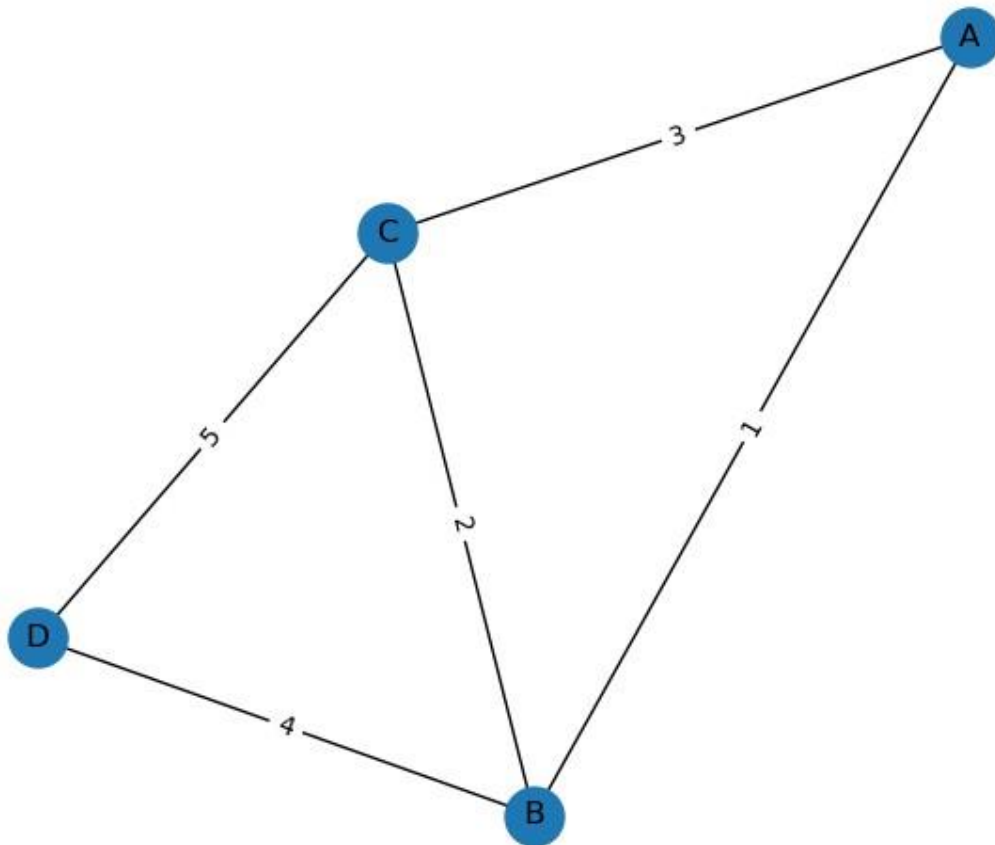


Output:

Input Graph Edges with Weights:

A -- B : 1
A -- C : 3
B -- C : 2
B -- D : 4
C -- D : 5

Input Graph



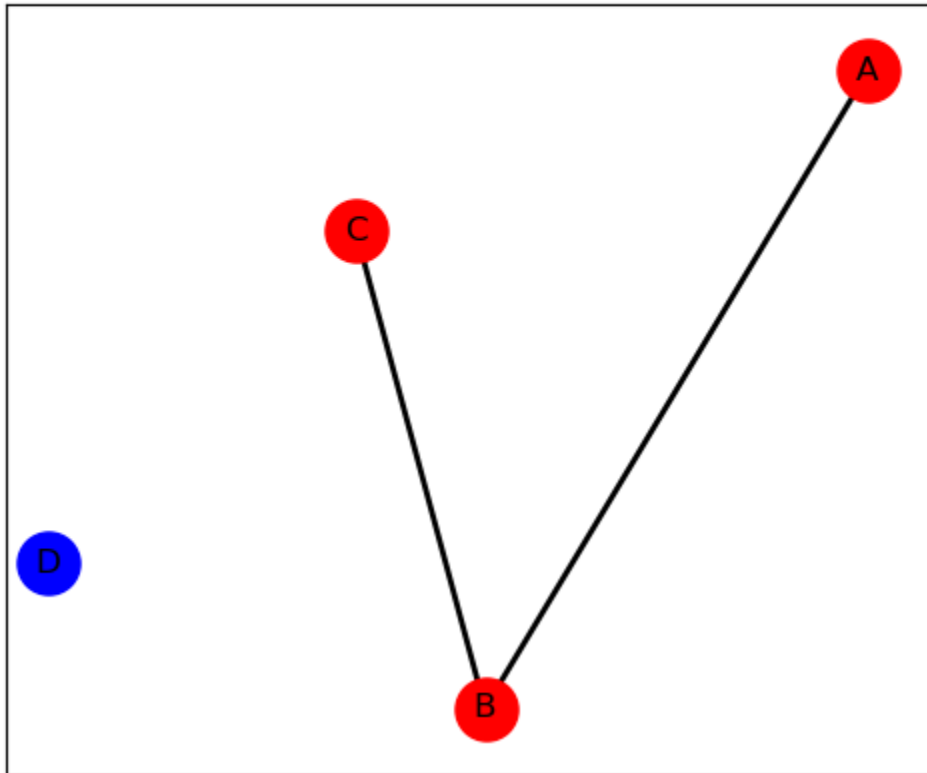


Clusters after cutting MST:

Cluster 1: ['A', 'B', 'C']

Cluster 2: ['D']

Clusters from MST



CONCLUSION

In this experiment, clustering was performed using the **Minimum Spanning Tree (MST)** approach. The process demonstrated how a weighted graph can be transformed into meaningful groups based on edge distances.

First, the MST was constructed by selecting edges with the smallest weights while avoiding cycles, ensuring that all nodes were connected with minimum total cost. This step effectively preserved the most important (shortest) relationships between data points.

Next, clustering was achieved by removing the largest edge in the MST. This edge represented the weakest connection in the structure. By cutting it, the graph split into separate connected components, which naturally formed clusters.